Is It Simplicity or Speed? A Controlled Test of Simplicity Bias

Oct 16, 2026 · @Jagriti

Summary in plain words

The question. Many papers say neural networks "prefer" simple features over complex ones, even when both are equally useful. This proposal asks whether that preference is really about *simplicity*, or whether the simple feature just wins because it is learned faster or has a bigger margin (more room between the classes).

The method. I build small synthetic data with two features that each predict the label perfectly. Feature A is simple: one cut-off separates the classes. Feature B is more complex: it has R regions that alternate between the classes. I train small networks and measure which feature the network actually uses. Then I tune B until it is learned as fast as A, and check whether the preference survives.

What the pilot run found. The pilot used 5 seeds and one data family, so it is only a first look. With no matching, the network used A and ignored B in every case, even when B was also a simple linear feature (R = 2). That cannot be about complexity. After speed matching, only 3 of the 8 matched cells were valid tests. In those, the gap between reliance on A and on B fell from about +0.5 to between -0.01 and +0.16. For R = 5 and R = 7, B could not be made as fast as A, so those cells say nothing yet.

What it means so far. The pilot supports a weaker claim: most of the unmatched preference comes from speed, margin or both. It cannot say whether a small direct effect of complexity remains, and it says nothing about simplicity bias in general.

Introduction

The claim under test is: *when several features can solve a task, neural networks preferentially learn the simpler ones.* If true, it helps explain shortcut learning, brittleness when the data shifts, and why a group of identically trained models can fail together (Shah et al., 2020). If it is only partly true, people may be fixing the wrong thing.

"Preferentially learn" can mean three different things:

1. **First.** The simple feature is learned earlier in training.

2. **At the end.** The trained network still relies on the simple feature.
3. **Exclusively.** The network ignores the complex feature completely.

These are not the same. An experiment can support the first and contradict the third, so a study has to say which one it tests. The claim also has a causal reading: the network prefers a feature *because it is simple*. That is the weakest part, for three reasons.

- **"Simple" has no single meaning.** Papers use linear separability, low frequency, low dimension, short description length and ease of optimization.
- **The simple feature differs in more than complexity.** In the datasets that support the claim, it is also usually learned faster, has a larger margin, and sits on a single input axis.
- **The evidence is narrow.** It comes mostly from one family of networks trained with SGD-style optimizers.

This study separates those factors with a small controlled test. It does not try to prove the claim wrong. It tries to find the conditions where the claim holds, weakens or reverses, and to say how large the effect of complexity is once speed and predictive strength are accounted for.

What the papers say

In short, my own understanding of the main paper: networks trained with standard procedures can lean heavily on simpler predictive features and underuse more complex, potentially useful ones. The limit is that this depends on how simplicity is defined and measured, and a preference may also come from predictive strength or ease of optimization rather than simplicity alone. The three challenge papers, in simple words:

Shah et al. (2020), *The Pitfalls of Simplicity Bias in Neural Networks*.

- *Main claim.* Networks trained with SGD and its variants can rely on the simplest predictive feature and ignore more complex ones, even when the complex ones are equally or more predictive.
- *What "simple" means.* The number of linear pieces a classifier needs to read the feature. A linear feature needs the fewest, and a slab feature needs more as it gets more slabs.
- *Evidence.* Synthetic datasets with a linear coordinate plus slab coordinates, and MNIST-CIFAR images (an MNIST digit stacked on a CIFAR image with a shared label). Randomizing the MNIST half destroys accuracy, while randomizing the CIFAR half

barely matters. The authors also report that the effect can persist when the simple feature is *less* predictive, that ensembles and adversarial training do not remove it, and they prove a version for one-hidden-layer ReLU networks.

- *Limit I use.* Everything is tied to one definition of simplicity, and the datasets do not separate simplicity from learning speed or margin.

Morwani et al. (2023), *Simplicity Bias in 1-Hidden Layer Neural Networks*.

- *Main claim.* A one-hidden-layer network can end up depending on only a low-dimensional projection of its input, even when richer directions exist.
- *What "simple" means.* Low dimension: the network is essentially a function of a few directions.
- *Evidence.* Theory for linearly separable data in the very wide (infinite-width) limit, plus experiments on Imagenette and Waterbirds-Landbirds. They also propose training successive models on features the earlier ones did not use, which made models more robust to Gaussian noise.
- *Limit I use.* The theory needs linear separability, one hidden layer and a specific regime, so how far it reaches is an open question.

Rahaman et al. (2019), *On the Spectral Bias of Neural Networks*.

- *Main claim.* ReLU networks fit low-frequency parts of a target function before high-frequency parts, so learning speed depends on frequency. The effect also depends on the shape of the data.
- *What "simple" means.* Low frequency (a smooth function that does not wiggle).
- *Evidence.* Fourier analysis of ReLU networks and synthetic experiments.
- *Limit I use.* It is about the *order of learning*, not about which feature a trained network relies on, and it uses a different idea of "simple" from Shah et al.

Other work that shapes the design.

- Kalimeris et al. (2019; also listed as Nakkiran et al.) find that early in training most of an SGD network's gain can be explained by a linear classifier, and that this linear part is kept later. This supports a speed-based reading.
- Pezeshki et al. (2021) describe *gradient starvation*: a strongly predictive feature shrinks the loss gradient and starves the others. This is a rival explanation based on predictiveness, not complexity.
- Hermann and Lampinen (2020) and Scimeca et al. (2022) find that the feature a network picks depends on how easy it is to extract *and* on how predictive it is, and that easy features can suppress harder ones, even when all cues are equally valid.

- Puli et al. (2023) show that shortcut use can come from the way cross-entropy training maximizes the margin, even with no extra information in the shortcut.
- Teney et al. (2024, *Neural Redshift*) argue that simplicity bias is not universal and depends on components such as ReLU. Tsoy and Konstantinov (2024) study it beyond linearly separable data. A NeurIPS 2025 paper on Adam and SGD reports that Adam resists simplicity bias better in a two-layer setting.
- Kirichenko et al. (2023) show that networks often encode the complex feature without using it, so "not used" and "not learned" must be measured separately.
- Qiu et al. (2024), cited in the original proposal, study how the complexity of a spurious feature changes what is learned. I did not check this paper myself.
- Teney et al. (2022) train diverse models to escape simplicity bias, and Chizat et al. (2019) describe lazy training, which is why width and output scale matter.

What is established, suggested, and proposed here. The papers establish that reliance on simple features happens in the datasets and regimes they test, and that learning speed depends on frequency. They suggest, but do not establish, that simplicity itself is the cause. Testing that causal step is what this study proposes to do.

Research gap and research question

In the studies above, the simple feature is also the one that is learned first, has the larger margin, and sits on one input axis. These things move together, so the data cannot say which one drives the preference. Shah et al. show the effect persists across different predictive strengths, but they do not equalize how fast each feature is learned. Hermann and Lampinen come closest by varying difficulty and predictiveness, but they focus on representations. I did not find a study that does a speed-matched test of reliance in a minimal setting, but my search was limited, so I claim no novelty beyond that.

Primary question. When two features predict the same label, and their learning speed and predictive strength are held equal, does the intrinsic complexity of a feature still decide which one a network relies on?

Three sub-questions follow:

1. **Exchange rate.** How much extra predictive strength or margin does a complex feature need to overtake a simple one, and does that change with the kind of complexity?
2. **Definition conflict.** When two ideas of "simple" (for example linear pieces and computational order) rank the same pair of features in opposite orders, which one predicts the network's choice?

3. **Robustness.** Does the answer change with the optimizer, architecture, starting weights or training length?

The pilot in this document touches only the primary question, with one data family and two optimizers.

Direct and total effect. If complexity acts only by making a feature slower to learn, then speed is a *mediator* (part of the path), not a confounder. So the design reports two numbers: the effect with speed matched (the direct effect of complexity) and the effect with speed free to vary (the total effect). The gap between them is the finding.

Why this question and not "is the simple feature learned first?" That question can be answered "yes" without telling us why. This one has different outcomes depending on the explanation.

Hypotheses, null hypothesis and rival explanations

How reliance is scored. Train a network on data where features A (simple) and B (complex) agree. Then test it on a *conflict set* where A and B point to opposite labels. The **preference index (PI)** is the fraction of conflict-set predictions that follow A, minus 0.5. So PI = +0.5 means pure reliance on A, 0 means no preference, and -0.5 means pure reliance on B. The pilot did not build a conflict set. It used a stand-in, the **gap**: accuracy lost when A is shuffled minus accuracy lost when B is shuffled. The gap also runs from about -0.5 to +0.5, so the same cut-offs apply.

Null hypothesis (H0). With learning speed, margin and label correlation of A and B matched, PI does not depend on the complexity gap between them. The mean PI lies inside an equivalence band of ±0.10 around zero for every complexity gap, optimizer and architecture tested.

My expectation (H1). The preference for the simple feature is mostly explained by speed and strength, not complexity. This agrees with H0 for small gaps on purpose, so the study can be wrong about it. Four predictions:

- **P1.** Unmatched, PI is at least +0.4, reproducing the literature.
- **P2.** Matched, the complexity effect shrinks by at least half, and PI stays inside ±0.10 for small gaps (R up to 3).
- **P3.** A leftover preference for the simple feature remains for large gaps (R of 5 or more), because those features need more capacity and more steps even at matched thresholds.

- **P4.** When two ideas of simplicity disagree, neither one alone predicts PI above chance, for both optimizers.

What would count against H1. PI above +0.10 at every matched gap, with the confidence interval excluding the band, would show a direct effect of complexity. A large PI that vanishes when the inputs are randomly rotated would show the effect was about axis alignment.

Statistics for the full study. At least 10 seeds per cell, bootstrap 95% confidence intervals, an equivalence test (TOST) so that "no effect" can be claimed and not just "not significant", and a Holm correction over the planned comparisons. The pilot used 5 seeds and none of these.

Rival explanations the design must tell apart from complexity.

1. **Optimization speed.** The simple feature is learned first, so the loss gradient shrinks and the complex feature is never learned. Complexity matters only through speed.
2. **Margin or size.** A larger margin or larger input values give larger gradients, so that feature wins by construction.
3. **Gradient starvation.** The feature more strongly tied to the label starves the other, whatever its complexity (Pezeshki et al., 2021).
4. **Coordinate alignment.** Networks are not rotation invariant. A feature on one input axis is easy to find at the start, and the same feature spread over many axes is not.
5. **Definition artifact.** "Linear versus slab" mixes linear separability, number of pieces and frequency, so a result may hold for one and not the others.
6. **Learned but unused.** The network may encode the complex feature and still not use it at the output (Kirichenko et al., 2023).
7. **Training protocol.** Short training, one optimizer, or a very wide (lazy) network may fix the apparent order without being a property of networks in general.

What "simple" means here

Idea of simple	How it is measured	Where it appears
Few linear pieces	Number of pieces needed (more slabs means more pieces)	Shah et al.
Low dimension	Rank of the learned first layer	Morwani et al.

Idea of simple	How it is measured	Where it appears
Low frequency	Fourier spectrum of the target	Rahaman et al.
Easy to optimize	Steps to reach a loss threshold	Kalimeris et al.; this study
Computational complexity	Parity order, circuit depth	This study (planned)

Used in the pilot. Simplicity is the number of regions R in feature B, following Shah et al. R = 2 is a linear control (one cut-off), and R = 3, 5 and 7 are progressively more complex. I chose it because it can be built exactly, it is the definition used by the paper being tested, and it gives a clean linear-versus-linear baseline.

Limits of this choice. More regions also means higher frequency, so a preference for A could mean "fewer pieces" or "lower frequency", and this design cannot separate the two. In the pilot, B also differs from A in margin and in size, which is a bigger problem (see the section on whether the run matches the plan). If a different definition would give a different winner, that is itself a qualification of the original claim. The full study adds parity and distributed-feature families and contradictory pairs to test exactly this.

Data: how the two features are built

This is the data generator in the notebook. Each example has 20 numbers: feature A, feature B and 18 distractor coordinates drawn from a standard normal distribution. The label is -1 or +1, and the classes are balanced.

- **Feature A (simple).** A equals the label times a random number between 0.5 and 1.5. The sign of A gives the class, and no point is closer than 0.5 to the boundary.
- **Feature B (R regions).** The B axis is cut into R regions, each of width s, side by side. Even-numbered regions belong to class +1 and odd-numbered regions to class -1. Points are drawn from the middle 70% of a region (15% is left empty at each edge) and then centred on zero. B's smallest distance to a boundary is therefore 0.15 s. With R = 2, B is a linear feature, just with a smaller margin than A.
- **Both features predict the label perfectly on their own.** Neither is made stronger by design.
- **Single-feature versions.** To measure how fast each feature is learned alone, the other feature's coordinate is replaced by noise.

- **Sizes in the pilot.** 1,000 training points and 4,000 test points.
- **The one knob, s.** In the unmatched condition s = 1. In the matched condition s is tuned so B alone is learned as fast as A alone. Note that s changes B′s size and margin together.

The full study adds a parity family and a distributed family (a linear feature spread over several rotated coordinates), and standardizes all inputs so scale cannot carry information.

Model and training, as run

- **Network.** A small multilayer perceptron (MLP) in PyTorch: 20 inputs, two hidden layers of 64 ReLU units, one output, with default initialization. The loss is binary cross-entropy.
- **Training.** Full batch (all 1,000 points at every step), 2,000 steps, no early stopping, no weight decay.
- **Optimizers.** Adam with learning rate 0.001, and "SGD" with learning rate 0.05 and momentum 0.9. Because training is full batch, the second one is really gradient descent with momentum, not stochastic.
- **Seeds.** 5 per cell. The data seed is 1000 plus the seed number, and the network starting weights use the seed number itself.
- **Speed measure.** The first step at which the training loss falls below 0.1 (2,000 if it never does).
- **Compute.** The whole grid took about 12 minutes on a Colab CPU.

The full study adds widths of 256 and 1024, tanh activations, a Fourier-feature input, an output-scaling parameter that moves the network between the rich and lazy regimes, mini-batch training, plain SGD, and training far past 100% accuracy.

How I measure which feature the model uses

Final accuracy cannot show which feature a model uses, because both features give perfect accuracy. So the study uses several measures, each with a stated weakness.

1. **Conflict-set preference index (primary, full study).** Test on examples where A and B point to opposite labels, and count how often the network follows A. It asks directly which feature wins. *Weakness:* conflict examples never appear in training, so this describes the learned function, not natural data.

2. **Ablation drop (used in the pilot).** Shuffle one feature across the test examples and see how much accuracy is lost. A big loss when A is shuffled and a small loss when B is shuffled means the network relies on A. If a network used both features equally, each drop would be about 0.25. *Weakness:* shuffled inputs are not like training data, and the result is read on the shuffled data only.
3. **Learning-order trajectory (full study).** Record both measures every 50 steps, to see which feature is used first and whether the order ever reverses. This separates "learned first" from "relied on at the end".
4. **Linear probes (full study).** Train a simple classifier on the last hidden layer to read out A and to read out B. High probe accuracy for B while the network follows A means B is *encoded but unused*, so "not learned" would be the wrong description.

Why not simply set a feature to zero. Zero is not a neutral value. For feature B it can fall inside a region of one class, so zeroing would push the prediction for a reason unrelated to reliance. Shuffling or sampling from the feature's own distribution avoids that.

Experiments: what changes and what stays fixed

Each experiment changes one thing. Runs are small, so any single run takes seconds on a laptop CPU. The full suite is about 1,500 to 2,000 runs, which is a few CPU-hours.

ID	Question	What varies	What stays fixed	Main readout	Status in the pilot
E0	Does the setup reproduce simplicity bias, and can speed matching be done?	Nothing (A-alone and B-alone runs, unmatched grid)	Architecture, optimizer	Steps to threshold; PI in unmatched runs	Done, using the shuffle gap instead of PI
E1	Does complexity matter once speed is matched?	Complexity gap (R, k, m); matched or unmatched	Margin family, optimizer, width	PI matched minus unmatched	Slab family only; valid for R = 2 and 3
E2	How much strength offsets complexity?	Complexity gap and B's margin or agreement, on a grid	Optimizer, width	Curve where PI = 0	Not run
E3	Do ideas of simplicity conflict?	Pairs ranked oppositely by two ideas	Strength matched across the pair	PI per pair and idea	Not run

ID	Question	What varies	What stays fixed	Main readout	Status in the pilot
E4	Does it depend on optimizer, architecture, regime or training length?	One factor at a time	Data, complexity gap	PI trajectory, probe accuracy	Optimizer only (Adam vs momentum GD)
E5	Can a complex feature be learned first?	B strongly predictive, A weakened by label noise	Complexity of B	Learning-order trajectory	Not run

Fixed in every pilot run: network (2 x 64 ReLU), 1,000 training points, 18 distractors, full batch, 2,000 steps, learning rate per optimizer. **Changed in the pilot:** R (2, 3, 5, 7), the scale s (1 or tuned), and the optimizer (Adam or momentum GD).

Controls and confounders

Each row names a rival explanation, the control that isolates it, the check that would show the control worked, and how far the pilot got.

Rival explanation	Control	Check that it worked	In the pilot
Optimization speed	Match steps-to-threshold of A alone and B alone within 10% by tuning margin or agreement, not input size; regression fallback where matching fails	Loss curves of A-alone and B-alone overlap; speed ratio near 1	Partly: matched by tuning s; valid only for R = 2 and 3
Margin or size	Standardize all inputs; report the margin ratio and put it in the regression	Equal input variance; margin ratio near 1 in matched cells	Not controlled: s changes B′s size and margin together
Gradient starvation (correlation strength)	Fix the agreement probability equal for A and B in E1; vary it on its own axis in E2	PI as a function of agreement probability at fixed complexity	Not run (both features are perfectly predictive)

Rival explanation	Control	Check that it worked	In the pilot
Coordinate alignment	Random rotation of the inputs; the distributed-feature family	PI before and after rotation; if the preference vanishes it was alignment	Not run
Definition artifact	Three complexity families and contradictory pairs (E3)	Whether the families give the same ordering of PI	Slab family only
Learned but unused	Linear probes on the last hidden layer; last-layer retraining	High probe accuracy for B while PI favors A means encoded but unused	Not run
Training protocol	Vary optimizer, width, activation, lazy scale and training length one at a time	PI stable or not across each factor	Optimizer only
Seed and data noise	At least 10 seeds, fresh data per seed, bootstrap intervals	Interval width compared with the ±0.10 band	5 seeds, standard deviation only
Off-distribution test artifact	Report both conflict-set PI and in-distribution ablation drop	The two measures rank cells the same way	Ablation drop only

Baselines. (a) A-alone and B-alone networks, which show what each feature costs in isolation. (b) Two linear features ($R = 2$, $k = 1$, $m = 1$), where any preference cannot be about complexity. (c) A random-label control for the probes. In the pilot, (a) and (b) are in place and (c) is not.

Regression fallback. Across the full grid, regress PI on complexity gap, log speed ratio, margin ratio and correlation ratio. The coefficient on complexity is then the direct effect even where matching was imperfect.

Pilot results

These are the numbers from the notebook (PyTorch, 5 seeds per cell, mean values). *Speed ratio* is the steps B alone needs divided by the steps A alone needs to reach loss 0.1, so 1.00 is a perfect match. A alone needed 64.5 steps with Adam and 36 with momentum GD. A matched cell counts as a valid test only if the speed ratio is within 10% of 1 and test accuracy is at least 0.9.

Optimizer	R	Condition	Scales	Speed ratio	Test accuracy	Drop if A shuffled	Drop if B shuffled	Gap (A minus B)	Valid test?
Adam	2	unmatched	1.00	1.28	1.00	0.48	0.02	+0.47	yes
Adam	2	matched	2.09	1.02	1.00	0.25	0.25	0.00	yes
Adam	3	unmatched	1.00	2.06	1.00	0.50	0.00	+0.50	yes
Adam	3	matched	27.92	1.16	1.00	0.06	0.44	-0.38	no (speed)
Adam	5	unmatched	1.00	3.05	1.00	0.50	0.00	+0.50	yes
Adam	5	matched	27.92	2.68	1.00	0.50	0.00	+0.49	no (speed)
Adam	7	unmatched	1.00	3.19	1.00	0.50	0.00	+0.50	yes
Adam	7	matched	27.92	10.19	1.00	0.50	0.00	+0.50	no (speed)
SGD	2	unmatched	1.00	1.58	1.00	0.50	0.00	+0.49	yes
SGD	2	matched	1.81	1.04	1.00	0.33	0.17	+0.16	yes
SGD	3	unmatched	1.00	4.22	1.00	0.50	0.00	+0.50	yes
SGD	3	matched	4.96	0.99	1.00	0.25	0.26	-0.01	yes
SGD	5	unmatched	1.00	9.26	1.00	0.50	0.00	+0.50	yes
SGD	5	matched	27.92	55.56	0.70	0.20	0.00	+0.20	no (speed, training failed)
SGD	7	unmatched	1.00	9.61	1.00	0.50	0.00	+0.50	yes
SGD	7	matched	27.92	55.56	0.51	0.00	0.00	0.00	no (speed, training failed)

"SGD" here is full-batch gradient descent with momentum. A speed ratio of 55.56 means B alone never reached the loss threshold in 2,000 steps. Standard deviations over seeds were 0.03 or less in every cell except SGD R = 5 matched (0.27, where training failed in some seeds).

What the numbers say, in plain words.

1. **Unmatched, the network uses only A, for every R and both optimizers.** Shuffling A costs 0.48 to 0.50 accuracy and shuffling B costs at most 0.02. This reproduces the literature, but the setup is confounded.
2. **At R = 2 both features are linear, yet the network still used only A.** A has the larger margin (0.5 against 0.15) and is learned faster (B alone needs 1.28 times as many steps with Adam and 1.58 times with momentum GD). That preference cannot be about complexity.
3. **Matching speed removes most of the gap, but not in a stable direction.** At R = 2, with speeds within 5%, the gap fell from about +0.47 to 0.00 (Adam) and to +0.16 (momentum GD). A speed difference of only 28% to 58% was enough to produce an almost total preference, so the network is very sensitive to small speed differences.
4. **At R = 3 with momentum GD, a 3-region feature is not disfavoured once speed is matched.** The speed ratio is 0.99 and the drops are 0.25 and 0.26. The Adam R = 3 cell shows strong reliance on B, but it misses the speed criterion (1.16) and needed a scale of 27.9, near the ceiling of 30, so I do not count it.
5. **R = 5 and 7 gave no valid test.** B could not be made as fast as A within the scale range, and training with momentum GD at scale 27.9 was unstable. The scale knob is also not reliably faster-with-bigger: for Adam at R = 7, B alone needed about 206 steps at s = 1 and about 658 steps at s = 27.9.
6. **The notebook's own checks.** C1, C2, C4 and C5 passed. C3 (matched R = 2: speed within 10% and more reliance on B than on A) did not pass. Adam gave equal use of both features and momentum GD still leaned toward A, so matching moved the network toward balance, not toward B.

Proportional reading. Of the three valid matched cells, none reproduces the unmatched gap of about +0.5, and the largest remaining gap toward A is +0.16. This supports the weaker claim that most of the unmatched preference comes from speed, margin or both. It cannot say whether a small direct effect remains, because the input-size confound is uncontrolled, only R up to 3 gave valid cells, and there are only 5 seeds. The original PDF also reports a second implementation in NumPy with its own initialization. That run is not in the notebook, so I could not check its numbers here.

Does the run match the plan?

The notebook follows the core design: two fully predictive features, speed matching, both optimizers, and a shuffle-based reliance measure. It is a smaller, simpler version of the full plan. The table lists each difference from my earlier draft (a Keras design) and from the full suite in the original PDF, and says whether it matters.

Item	Earlier draft / full plan	What the notebook did	Does it matter?
Library	Keras	PyTorch	No. PyTorch is also what the challenge brief prefers
Network	2 x 100 ReLU	2 x 64 ReLU (a 2 x 128 setting exists but was not run)	Minor for a pilot
Inputs	10 (8 distractors)	20 (18 distractors), as in the PDF plan	Harder task; fine
Data size	10,000 train	1,000 train, 4,000 test	Smaller pilot; noisier estimates
Training	Mini-batch 128, 200 epochs, plain SGD	Full batch, 2,000 steps; "SGD" has momentum 0.9	**Yes.** It is gradient descent, not stochastic, so mini-batch noise is untested. A 2026 paper (Mirzaie et al.) reports the shortcut-reducing effect comes from mini-batch noise and that full-batch descent does not share it
Seeds and statistics	10 seeds, 95% intervals, equivalence band	5 seeds, standard deviation only	**Yes.** No claim of "no effect" can be made yet

Item	Earlier draft / full plan	What the notebook did	Does it matter?
Feature B	5 regions inside a fixed range	R = 2, 3, 5, 7 at a fixed region width, so the range grows with R	An improvement: R = 2 is a clean linear-versus-linear control, and a fixed width keeps B′s margin the same across R
Reliance measure	Conflict set (primary), replacement (secondary), confidence check	Shuffle drop only, which is close to my secondary measure	**Yes.** The gap is only a stand-in for PI, and the primary measure is missing
Speed matching	B alone reaches 95% training accuracy within 20% of A alone, on a grid of scales	Steps to training loss below 0.1, bisection on log scale in [0.3, 30] (5 steps, 2 seeds), 10% tolerance checked afterwards	Same idea and finer. But the knob s changes B′s size and margin together, and speed is not always faster with larger s, so the bisection can fail silently. The 27.9 values are the ceiling
Size confound	Report s next to every matched result	Reported, but not controlled	**Yes.** It is the main confound; the full plan standardizes inputs and tunes margin instead
Training-dynamics checkpoints	Reliance at 8 points in training	Final model only	**Yes.** "Learned first" cannot be separated from "relied on at the end"
Learning-rate check	One lower and one higher rate per optimizer	Not run	Minor for now

Item	Earlier draft / full plan	What the notebook did	Does it matter?
Pre-set checks	Outcome table written before running	Checks C1 to C5 written in the notebook, and C3 failed	Good practice. C3 failing is a result, not an error

Verdict. The experiments you ran fit the plan's core logic and use the same numbers as your PDF (I checked its PyTorch table against the notebook output and they agree). They are an early pilot: two things in the plan are missing (the conflict-set measure and the checkpoints), one is only partly controlled (size and margin), and one differs in a way that changes the meaning of "SGD" (full batch with momentum).

What each possible result would mean

No outcome is assumed. This table says in advance what each result would mean, so the conclusion stays in proportion to the evidence, and the last column shows what the pilot has seen so far.

Possible result	Interpretation	Pilot so far
PI large unmatched, near zero matched (my expectation)	The effect in the literature is largely speed or margin, not complexity; simplicity bias needs qualifying	Fits Adam at R = 2 (+0.47 to 0.00) and momentum GD at R = 3 (+0.50 to -0.01); momentum GD at R = 2 ends at +0.16
PI stays above +0.10 at every matched gap, interval excluding the band	Complexity has a direct effect; the original claim survives this test	Not seen. Only one valid cell is above +0.10, and with 5 seeds
PI shifts toward the complex feature when it is made stronger (E2)	Data correlations can overpower simplicity; the exchange rate gives the size of the effect	Not run
PI reverses or is large for two linear features	The preference is about margin, size or alignment, since complexity is identical	Yes: at R = 2 the network used only A (+0.47 and +0.49)

Possible result	Interpretation	Pilot so far
PI differs between Adam and SGD	Optimization, not architecture, is responsible	Possible at R = 2 matched (0.00 against +0.16), but 5 seeds cannot settle it
PI disappears under random rotation	The effect was axis alignment, not complexity	Not run
Fourier-feature inputs remove the preference	The preference depends on the spectral bias of the input encoding	Not run
Different complexity families give different PI orderings (E3)	"Simplicity" is not one thing; the claim must name its definition	Not run
Probe accuracy for B is high but PI favors A	The complex feature is learned but unused	Not run
PI drifts toward B with longer training	The apparent preference is a transient of learning order	Not run (final model only)
B cannot be matched to A's speed	The question cannot be answered with this setup; report the limit	Yes for R = 5 and 7 with both optimizers
No effect found at any tested setting	A failure to falsify under these conditions; it says nothing about natural images or large models	Not applicable yet

A mixed result is the most likely one: a preference present in some settings and absent in others. Reporting where the boundary lies is the aim.

If the results support the claim. I would report that I tried to falsify it and could not under these conditions, name the conditions (definition of simplicity, model, optimizer, data), and avoid saying simplicity bias has been proved. A confirmation with strong controls is a finding, not a failure.

Limitations

What this study cannot establish.

- It says nothing directly about natural images, language or large networks. Synthetic features are independent and axis-aligned, and real features are neither.
- A null result at the tested settings is not proof that complexity never matters. It is a failure to falsify under those conditions.
- The conflict set is out of distribution, since no training example has A and B disagreeing. Reliance measured there describes the learned function, not training dynamics.

Variables that stay confounded.

- **Speed may be a mediator.** If complexity makes a feature slow, matching speed removes part of the very effect of complexity. The design reports matched and unmatched results for this reason, but it cannot say how much of the total effect counts as genuine simplicity bias.
- **Matching is approximate.** Speed is measured at a threshold and averaged over a few seeds, so matched cells differ by up to 10%. The regression fallback relies on a roughly linear model of the confounds.
- **Margin, agreement and speed are correlated.** Tuning one moves the others, so a clean single-variable change is not always possible.
- **The definition of simplicity is incomplete.** The suite covers three ideas (linear pieces, parity order, distributed dimension) and one operational one (speed). It leaves out description length, Kolmogorov complexity, frequency measured on a data manifold, and any idea defined on learned representations.

Pilot limits.

- Five seeds, width 64, 1,000 training points, no standardized inputs, no conflict set and no checkpoints.
- Calibration failed for R of 5 or more, and training with momentum GD became unstable at large scale.
- "SGD" is full-batch gradient descent with momentum.
- The shuffle-drop measure shows reliance only on shuffled inputs.
- Because the matched R = 2 result differs in sign between the PyTorch run and the NumPy run reported in the PDF, a small leftover gap should not be read unless it holds across both.

Future experiments. Move to image-based features (a coloured-background and shape benchmark such as the MNIST-CIFAR variants), add convolutional networks and transformers, measure learning dynamics with NTK alignment or per-feature gradient

norms, and test whether the exchange rate from E2 predicts behaviour on a real spurious-correlation benchmark.

Conclusion

This study cannot show that simplicity bias is absent, and it is not built to. It is built to say *how much* of the preference for simple features comes from complexity itself, as opposed to learning speed, margin, correlation strength and axis alignment, and under which optimizers and architectures that share changes.

Three outcomes would be informative. If the preference shrinks a lot once speed and strength are matched, the original claim should be restated as a claim about optimization ease. If it survives matching and rotation, simplicity bias has a direct component, and the exchange rate from E2 measures its size. If different ideas of simplicity give different answers, the claim needs a stated definition before it can be tested at all.

The pilot already shows why this matters. With no matching, a network preferred one linear feature over another linear feature, so that result could not have been about complexity. Once speed was matched, the gap fell from about +0.5 to between -0.01 and +0.16 in the three valid cells. That supports the weaker claim that most of the preference comes from speed or margin, and it leaves open whether a small direct effect remains.

A result of "I tried to falsify the claim under these conditions and could not" is a valid and useful outcome of this design.

Next steps and README

Next steps, in order.

1. Standardize all inputs and tune B's margin instead of its scale, so size cannot carry information.
2. Add the conflict-set preference index next to the shuffle drop.
3. Raise the seeds to 10 and add bootstrap intervals and the equivalence test.
4. Replace or widen the knob so R = 5 and 7 can be matched, or report that they cannot.
5. Record PI every 50 steps to separate "learned first" from "relied on at the end".
6. Add mini-batch SGD without momentum and a second learning rate for each optimizer.
7. Then run E2 and E3 (exchange rate and conflicting definitions).

README for the notebook.

- **Purpose.** A pilot of speed-matched simplicity-bias tests in a small MLP.
- **Needs.** Python 3, PyTorch, NumPy, pandas and matplotlib.
- **How to run.** Open the notebook in Colab and run the cells in order. `PILOT = True` (the setting used here) takes about 12 minutes on a CPU. `PILOT = False` uses a wider network (128), 2,000 points, 2,500 steps and 7 calibration steps; it was not run.
- **Cells.** Settings, data generator, model/training/reliance functions, speed matching (bisection on log scale), the grid run, the summary table, the pass/fail checks C1 to C5, and the bar plot (invalid matched cells are marked with a cross).
- **Seeds.** Data seed is 1000 plus the seed number, network seed is the seed number, and the calibration runs use seeds 100 and 101.
- **Output.** `results.csv` with one row per run: optimizer, R, condition, scale, steps for A, steps for B, seed, accuracy, drop if A shuffled, drop if B shuffled.
- **How to read it.** The gap is the drop for A minus the drop for B. A value near +0.5 means the network used only A, near 0 means it used both, and negative means it leaned on B. Only matched cells whose speed ratio is within 10% of 1 and whose accuracy is at least 0.9 are valid tests.

References

1. Shah, H., Tamuly, K., Raghunathan, A., Jain, P., and Netrapalli, P. (2020). The Pitfalls of Simplicity Bias in Neural Networks. NeurIPS 2020. arXiv:2006.07710
2. Morwani, D., Batra, J., Jain, P., and Netrapalli, P. (2023). Simplicity Bias in 1-Hidden Layer Neural Networks. NeurIPS 2023. arXiv:2302.00457
3. Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., and Courville, A. (2019). On the Spectral Bias of Neural Networks. ICML 2019. arXiv:1806.08734
4. Kalimeris, D., Kaplun, G., Nakkiran, P., Edelman, B., Yang, T., Barak, B., and Zhang, H. (2019). SGD on Neural Networks Learns Functions of Increasing Complexity. NeurIPS 2019. arXiv:1905.11604
5. Pezeshki, M., Kaba, S.-O., Bengio, Y., Courville, A., Precup, D., and Lajoie, G. (2021). Gradient Starvation: A Learning Proclivity in Neural Networks. NeurIPS 2021. arXiv:2011.09468
6. Hermann, K. L., and Lampinen, A. K. (2020). What Shapes Feature Representations? Exploring Datasets, Architectures, and Training. NeurIPS 2020. arXiv:2006.12433

7. Scimeca, L., Oh, S. J., Chun, S., Poli, M., and Yun, S. (2022). Which Shortcut Cues Will DNNs Choose? A Study from the Parameter-Space Perspective. ICLR 2022. arXiv:2110.03095
8. Puli, A. M., Zhang, L. H., Wald, Y., and Ranganath, R. (2023). Don't Blame Dataset Shift! Shortcut Learning due to Gradients and Cross Entropy. NeurIPS 2023. arXiv:2308.12553
9. Teney, D., et al. (2024). Neural Redshift: Random Networks are not Random Functions. arXiv:2403.02241
10. Tsoy, N., and Konstantinov, N. (2024). Simplicity Bias of Two-Layer Networks beyond Linearly Separable Data. arXiv:2405.17299
11. The Rich and the Simple: On the Implicit Bias of Adam and SGD. NeurIPS 2025. arXiv:2505.24022 (authors not recorded here)
12. Mirzaie, N., et al. (2026). Implicit Regularization of SGD Reduces Shortcut Learning. ICLR 2026. (I read only a summary of this paper.)
13. Kirichenko, P., Izmailov, P., and Wilson, A. G. (2023). Last Layer Re-Training is Sufficient for Robustness to Spurious Correlations. ICLR 2023.
14. Qiu, G., Kuang, D., and Goel, S. (2024). Complexity Matters: Feature Learning in the Presence of Spurious Correlations. ICML 2024. (Cited in the original proposal; not checked.)
15. Teney, D., Abbasnejad, E., Lucey, S., and van den Hengel, A. (2022). Evading the Simplicity Bias: Training a Diverse Set of Models Discovers Solutions with Superior OOD Generalization. CVPR 2022.
16. Chizat, L., Oyallon, E., and Bach, F. (2019). On Lazy Training in Differentiable Programming. NeurIPS 2019.